Rapport d’avancement

Antoine Bernard
Antoine Mercier

Table des matières:

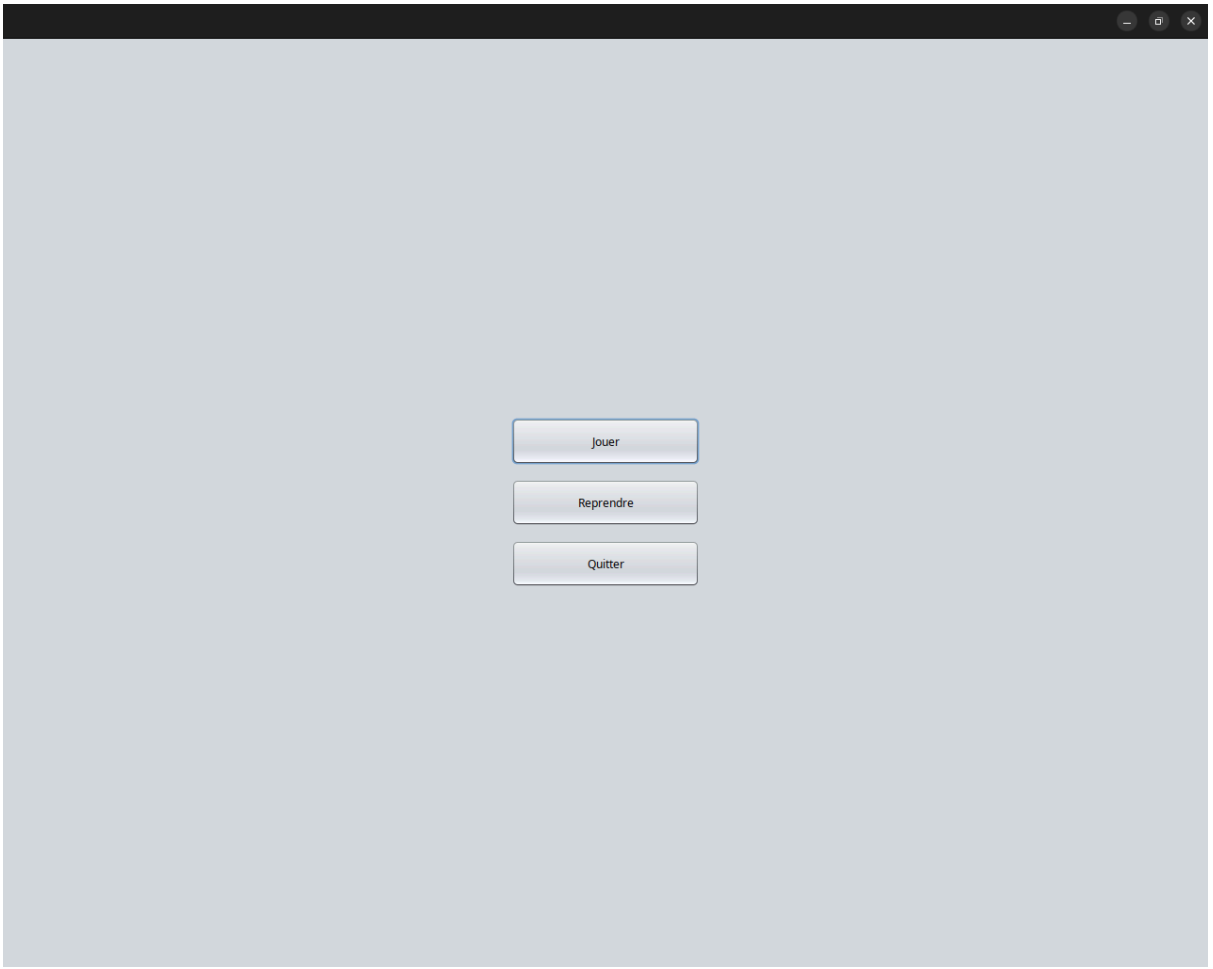
1.Introduction :
2.Fonctionnalité de notre programme :
3. Structure de notre programme :
4.Mécanisme de sauvegarde d’une partie :
5. Exposition de l'algorithme de révélation :
6. Conclusion :

1.Introduction :

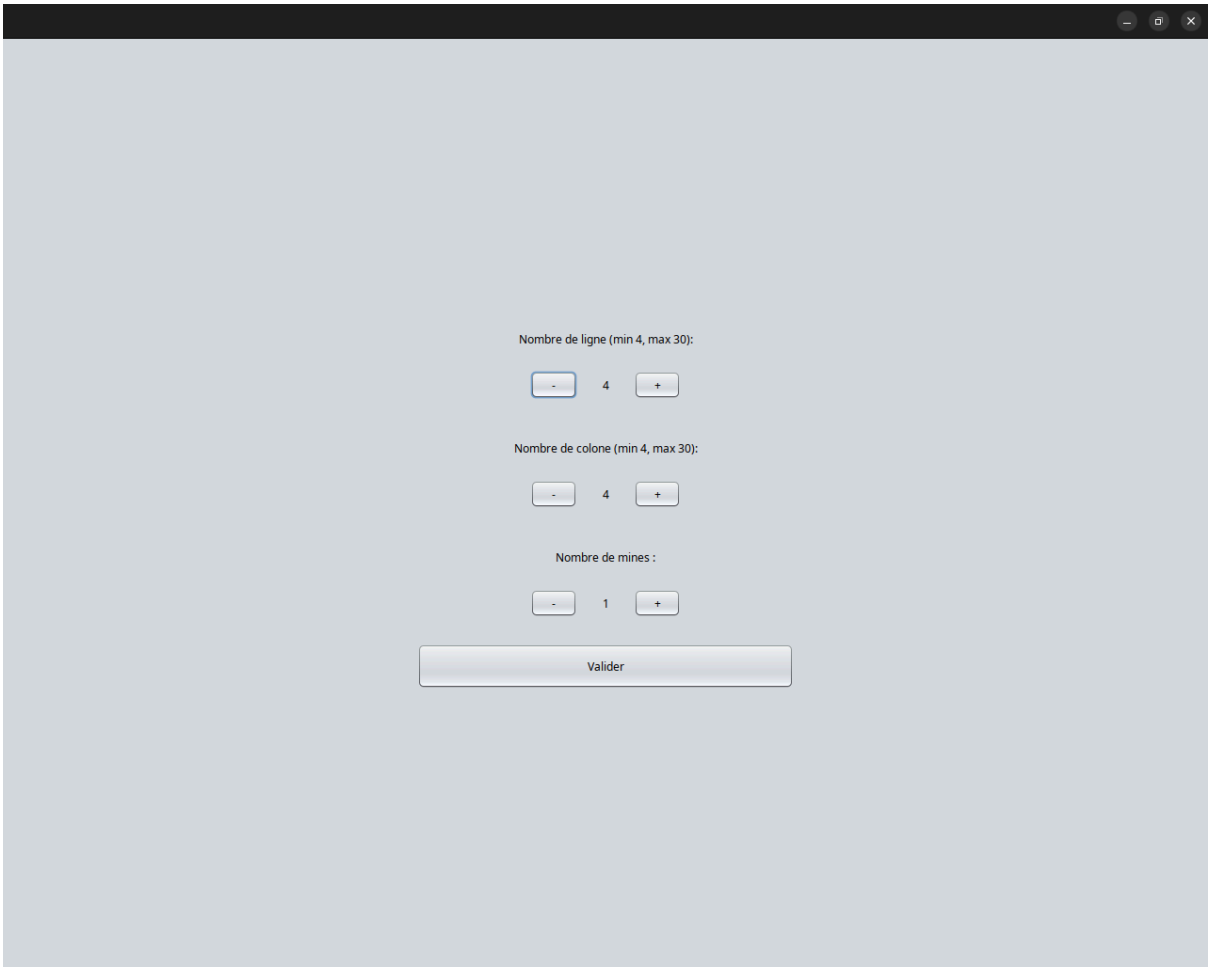
Nous avons réalisé un jeu de démineur ou le joueur choisit son nombre de ligne et colonne (limité entre 4 et 30) avec la possibilité de sauvegarder une partie pour la reprendre ensuite.

2.Fonctionnalité de notre programme :

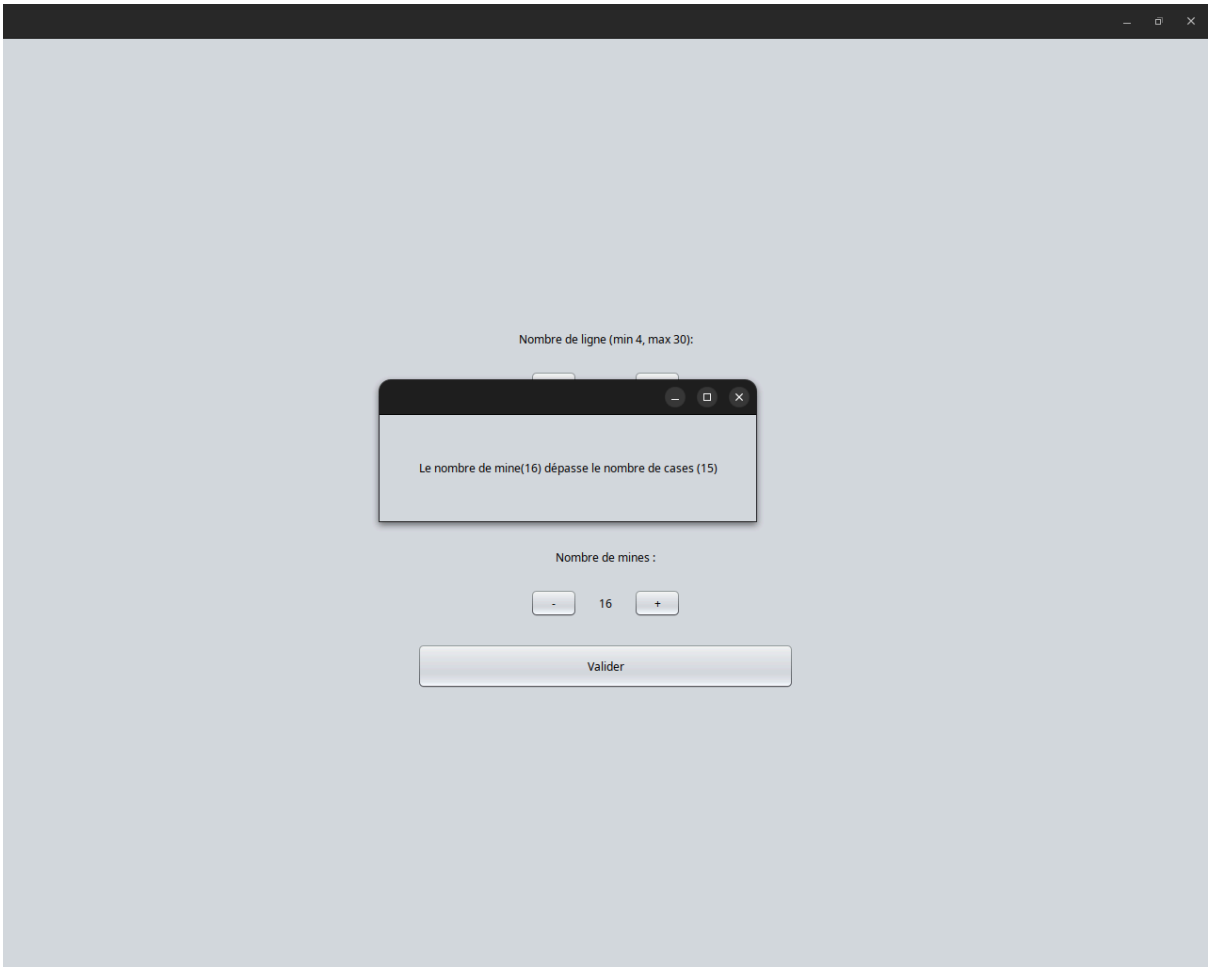
Lorsque l'on lance notre jeu, on arrive sur cette page avec trois boutons. Le bouton “Jouer" permet d’accéder à la page de choix du nombre de cases et du nombre de mines. Le bouton “Reprendre” permet de reprendre une partie en cours que l’on a sauvegardée. Le bouton “Quitter” permet de quitter le jeu.



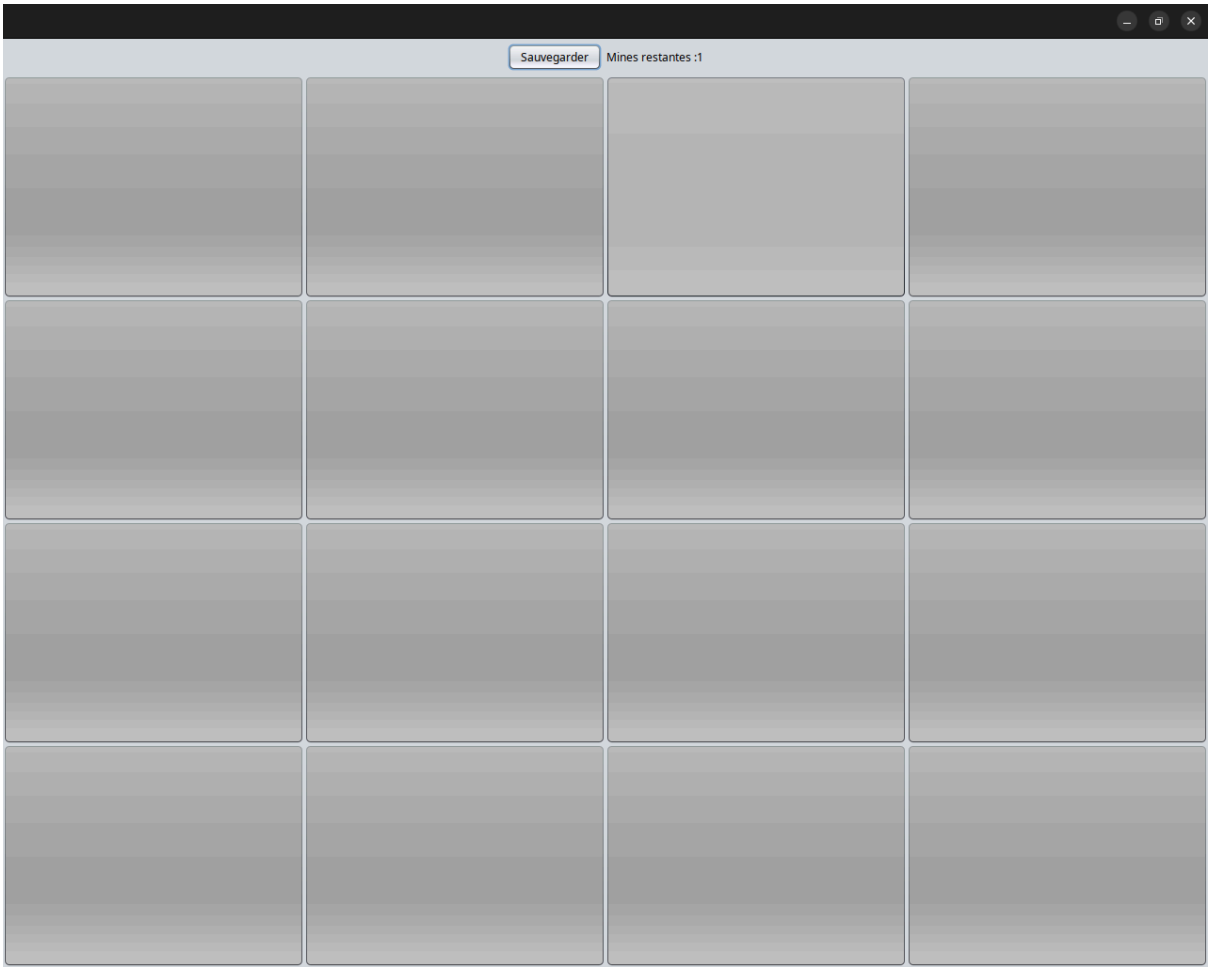
Lorsque l'on appuie sur le bouton "Jouer", cela nous amène à la page du choix du nombres lignes, de colonnes et de mines.
On peut choisir le nombre de lignes et de colonnes en appuyant sur le bouton "+" (incrémente) et "-" (décrémente). Lorsque l'on atteint 30 et que l'on appuie sur "+" cela nous ramène à 4. De même, lorsque l'on est à 4 et que l'on appuie sur "-" cela nous ramène à 30. Le nombre de lignes et de colonnes n'a pas besoin d'être obligatoirement identique.
Ensuite, il y a le choix du nombre de mines. Cela fonctionne de la même manière que pour les lignes et les colonnes, en utilisant les boutons "+" et "-". Cependant, le nombre est bloqué lorsqu'on tente de dépasser le nombre total de cases (lignes x colonnes), et il n'est pas possible de descendre en dessous de 1.



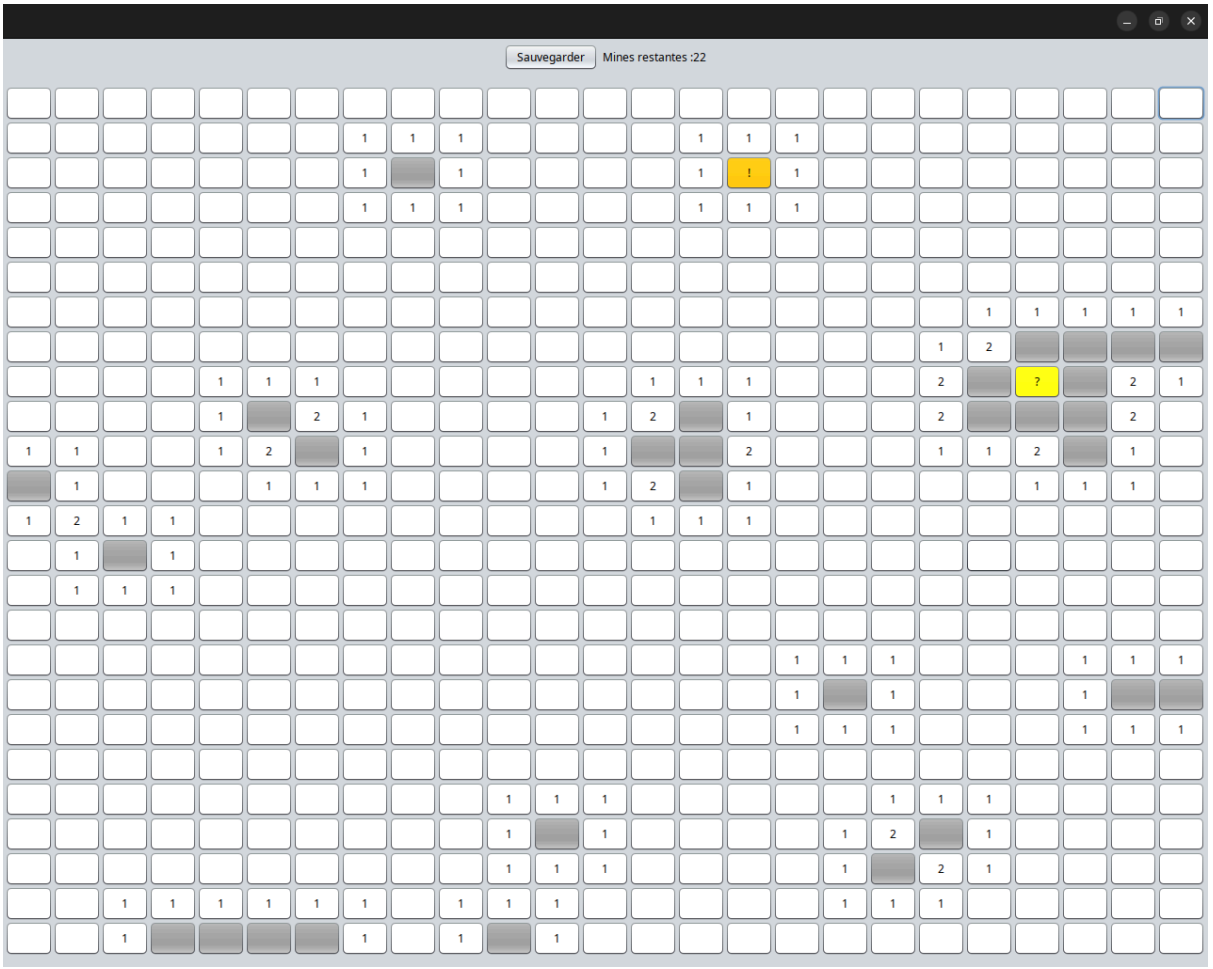
Lorsque l’on appuie sur le bouton “Valider”, cela nous amène à la page de jeu. Cependant, si le nombre de mines est supérieur au nombre de cases, un message s’affiche d’erreur s’affiche car il ne peut pas y avoir plus de mines que de cases.



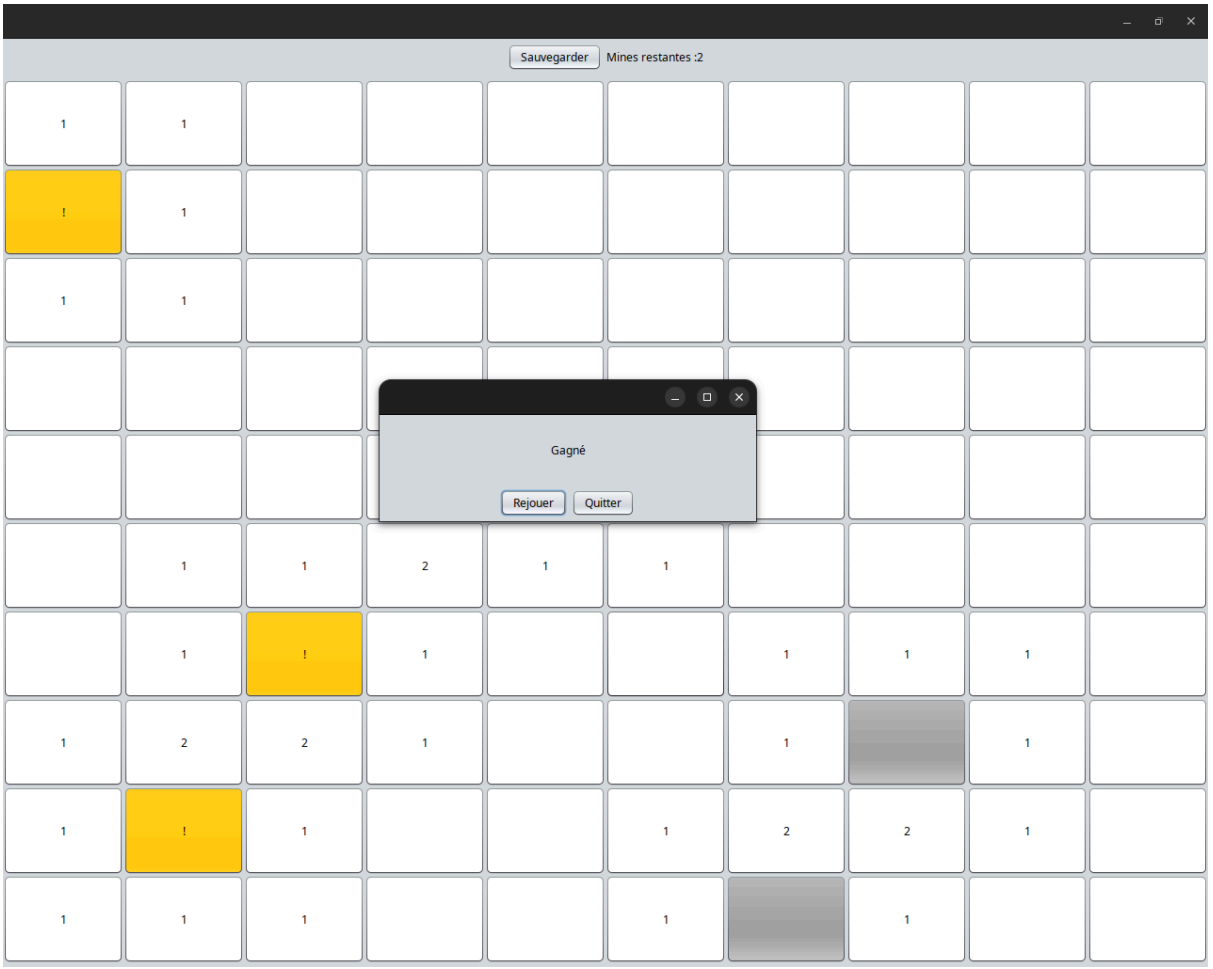
Maintenant que le choix est fait, on arrive sur la page de jeu.
En haut il est possible de sauvegarder la partie avec le bouton “Sauvegarder” et cela ferme également le jeu. On voit également le nombre de mines restantes à trouver. Ce nombre diminue seulement lorsque l’on est certain qu’une case est minée (celle marqué par “!”) mais il ne diminue pas lorsqu’on a un simple soupçon (celle marqué par “ ?”).
Ensuite, en bas, on trouve la grille de jeu, sur laquelle on peut effectuer un clic gauche pour révéler définitivement une case ou un clic droit : une première fois pour être certain, une deuxième fois lorsque l’on a un doute et une troisième fois pour revenir à une case non révélée.



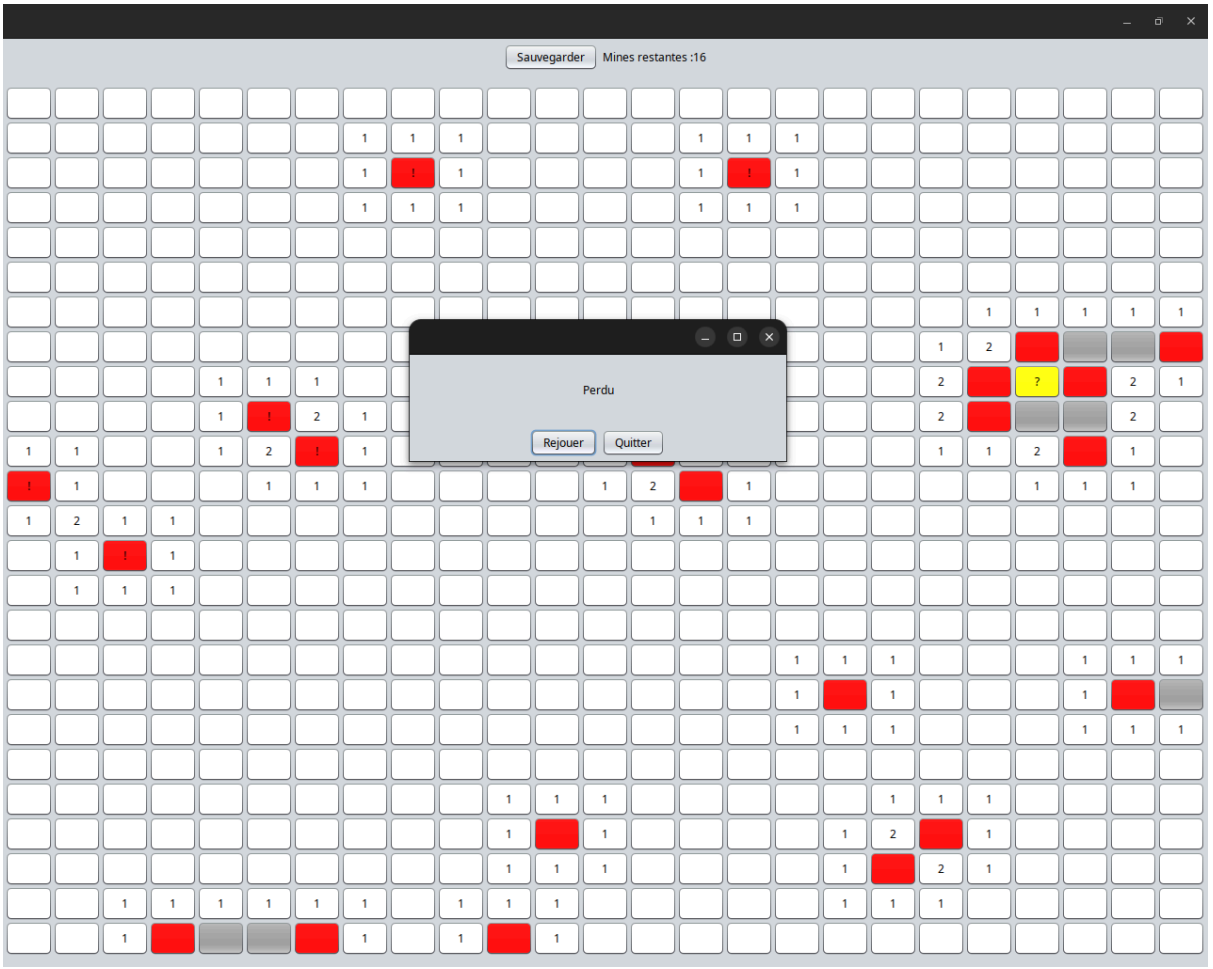
Ici, on peut voir le déroulement d'une partie en cours, avec les cases grises non révélées, les cases blanches révélées sans mines autour, ainsi que les cases blanches avec des numéros indiquant le nombre de mines autour.
On peut également voir les cases marquées comme suspectes (avec “?” et jaune), ainsi que celles pour lesquelles on est certain de la présence d’une mine (avec “!” et orange).



Lorsqu'on a gagné, une petite fenêtre apparaît pour indiquer que l’on a gagné.
Il est alors possible de choisir de recommencer une partie grâce au bouton “Rejouer”, ce qui ferme la partie finie et nous ramène à la page d’accueil ou bien de quitter le jeu avec le bouton “Quitter”.



Lorsqu'on a perdu, une petite fenêtre apparaît pour indiquer que l'on a perdu et les cases minés apparaissent en rouge.
Il est alors possible de choisir de recommencer une partie grâce au bouton "Rejouer", ce qui ferme la partie finie et nous ramène à la page d'accueil ou bien de quitter le jeu avec le bouton "Quitter"..



3. Structure de notre programme :

Le programme est structuré en plusieurs classes ayant chacune une responsabilité spécifique, ce qui permet de séparer la logique du jeu, l'interface graphique et la gestion des interactions.

La classe "Grille" est le cœur du programme. Elle contient l'ensemble des données du jeu, notamment une tableau à 2 dimensions d'objets "EtatCase", représentant l'état de chaque case (mine, nombre de mines autour, état visible ou non). Elle gère également les règles du jeu, comme le placement des mines, le calcul des cases adjacentes ou encore la gestion de la fin de partie.

La classe "EtatCase" représente une case du jeu du point de vue logique. Elle encapsule les différentes informations nécessaires (présence d'une mine, état révélé, marqué ou soupçonné) ainsi que les méthodes permettant de modifier cet état.

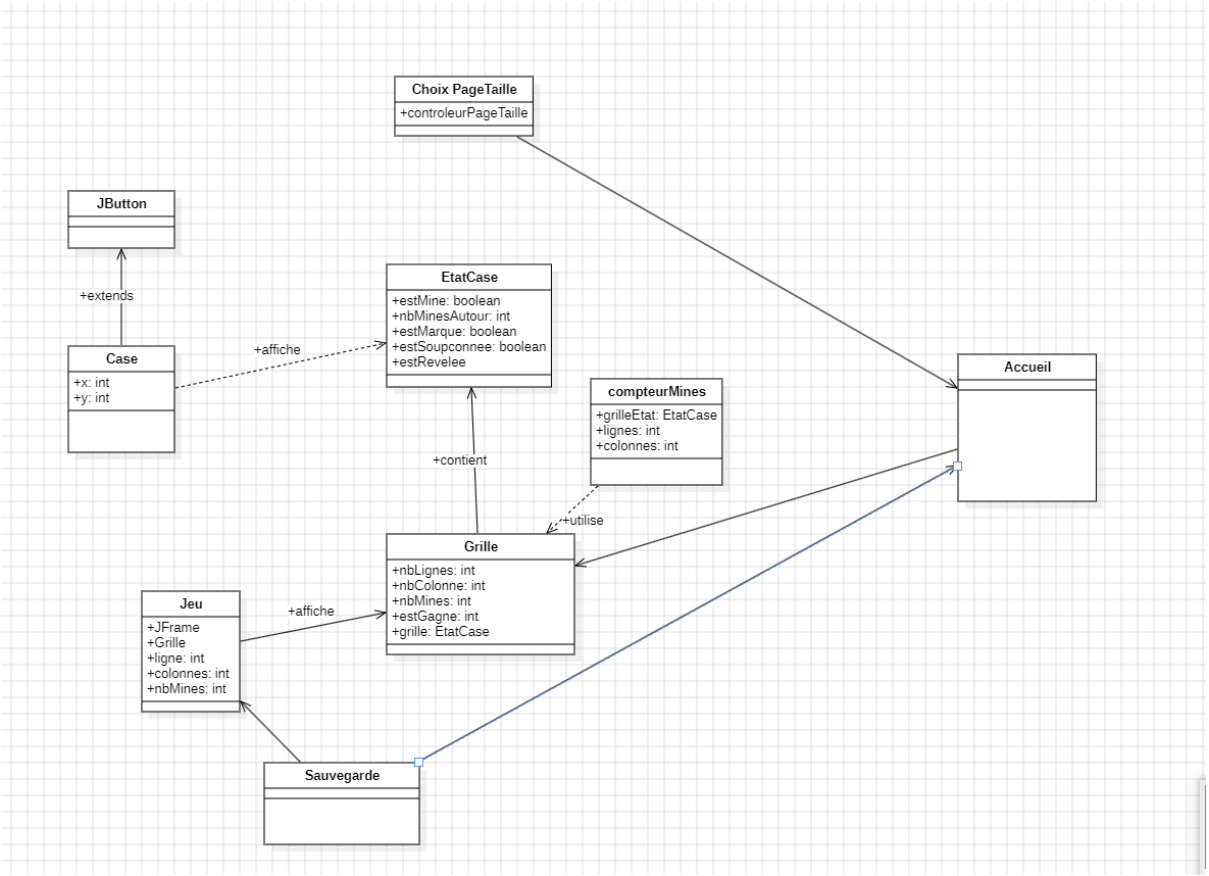
La classe "Case" correspond à la représentation graphique d'une case, héritant de JButton. Elle est utilisée pour afficher l'état de chaque case à l'écran.

Les interactions utilisateur sont gérées par des contrôleurs, comme la classe "ControleurCase", qui réagit aux actions de l'utilisateur (clics) et met à jour la grille en conséquence.

La classe "Jeu" est responsable de l'initialisation et de l'affichage de l'interface. Elle construit la fenêtre, organise les composants graphiques et assure la mise à jour de l'affichage en fonction de l'état de la grille, que ce soit pour une nouvelle partie ou une partie chargée.

Enfin, la classe "Sauvegarde" permet d'enregistrer et de restaurer une partie en cours à partir d'un fichier .dat, généré quand on quitte une partie. Elle gère la conversion entre l'état interne de la grille et une représentation externe stockée dans un fichier.

Ainsi, l'organisation du programme repose sur une séparation entre la logique du jeu, l'affichage et la gestion des interactions, ce qui facilite la compréhension et la maintenance du code.



4.Mécanisme de sauvegarde d'une partie :

Le mécanisme de sauvegarde repose sur une écriture et une lecture manuelle d'un fichier texte en .dat contenant l'état complet de chaque cases de la grille de jeu.

Lors de la sauvegarde, l'état des cases de la partie est écrit dans un fichier. La première ligne contient les dimensions de la grille et le nombre de mines, ce qui permet de reconstruire correctement la structure de la partie sauvegardée lors du chargement. Ensuite, chaque ligne du fichier correspond à une ligne de la grille.

Pour chaque case, plusieurs informations sont enregistrées sous forme de valeurs séparées : la présence d'une mine, le nombre de mines adjacentes, et les différents états possibles de la case (révélée, marquée ou soupçonnée). Ces
valeurs sont écrites en 0 ou 1 (pour les booléens) et séparées par des espaces, tandis que les cases sont séparées par un "|".

Lors du chargement, le fichier est lu ligne par ligne. La première ligne permet de recréer une grille avec les bonnes dimensions. Ensuite, chaque ligne est découpée pour extraire les informations de chaque case. Ces données sont donc parsées et mise dans les objets "EtatCase" correspondants afin de restaurer l'état exact de la partie.

Une fois la grille reconstruite, elle est transmise à la classe "Jeu", qui se charge de reconstruire l'interface graphique à partir de ces données, notamment via une méthode "chargerPartie" dédiée au chargement d'une partie.

Ce mécanisme permet de sauvegarder et restaurer une partie de manière fiable. Il y a toutefois un inconvénient, c'est le fait que les états de marquage utilisent un emplacement pour chaque état (marquée, révélée, soupçonnée), et vu qu'ils ne peuvent pas être superposés, on aurait pu utiliser un int qui associe pour chaque valeur un état, dans cette configuration le fichier aurait été plus léger. Mais la méthode actuelle fonctionne donc nous avons préféré garder plutôt que refaire tout le programme.

5. Exposition de l'algorithme de révélation :

L'algorithme de révélation des cases repose sur trois cas principaux. Lorsqu'une case cliquée contient une mine, la partie est perdue et toutes les mines sont révélées. Si la case ne contient pas de mine mais possède des mines adjacentes, seule cette case est révélée et le nombre correspondant est affiché.

En revanche, si la case ne contient aucune mine adjacente, un algorithme récursif DFS est utilisé. Celui-ci consiste à révéler la case puis à explorer récursivement ses voisines. L'exploration ne se poursuit que pour les cases valides, non minées et non encore révélées.

Ce mécanisme permet de révéler automatiquement des zones entières du plateau, reproduisant le comportement attendu du démineur.

6. Conclusion :

Antoine BERNARD:

Un projet vraiment intéressant pour comprendre comment fonctionne un jeu du type du démineur. La logique du code change radicalement par rapport à celle du projet du taquin

parce qu'ici il y a une vraie gestion de l'état de chaque case . Je me suis principalement occupé du code de la grille, des parties avec les intéractions entre les cases (cachée, marquée, soupçonnée et minée) et des sauvegardes et de chargement de partie.

J'ai aussi essayé au maximum de respecter la MVC mais la séparation des contrôleurs entre les vues et de la logique à été trop complexe à maintenir jusqu'au bout du projet, donc j'ai préféré distinguer clairement la logique du reste, ainsi des classes ne gèrent que la logique mais les vues et les contrôleurs sont parfois mélangés. Ce compromis permet de conserver un code fonctionnel, l'affichage et la logique modifiables assez facilement mais des contrôleurs qui demandent un peu plus de compréhension.
J'ai aussi fais en sorte de gérer les erreurs autour du fichier de sauvegarde pour éviter que le programme plante si le fichier à un problème.

Antoine MERCIER :
Durant ce projet je me suis principalement occupé de l'interface et de l'interaction avec l'utilisateur. J'ai conçu les fenêtres et les messages qui guident le joueur : la victoire, la défaite mais aussi les messages d'erreurs dans le choix du nombre de mines.
J'ai également mis en place les options permettant de choisir le nombre de lignes, de colonnes et de mines.
Cependant, j'ai rencontré plusieurs difficultés. J'ai eu du mal à limiter la portée de mes variables, qui se retrouvaient parfois utilisées un peu partout dans ce projet, ce qui rendait le suivi du code difficile. De plus, la logique des constructeurs et la manière de les appeler entre les différents fichiers n'ont pas toujours été fluide, créant parfois des dépendances importantes entre mes classes. Enfin, j'ai eu du mal à bien séparer la logique pure du jeu de la partie graphique et à répartir efficacement le code dans différents fichiers.
Cela m'a permis de comprendre que la réussite d'un projet n'est pas seulement dans le fait qu'il "marche", mais surtout dans la mise en place d'un code clair et structuré pour qu'il soit compréhensible.